MERN Stack — Complete Interview Q&A;

All questions from the image answered with examples. Sections: MongoDB · Express.js · React.js · Node.js · JS General · Full Stack/MERN · Tools & Concepts

MONGODB

1. What is MongoDB?

A NoSQL document database that stores data as JSON-like BSON documents instead of rows/tables. Schema-less, highly scalable, good for unstructured or evolving data.

2. Features of MongoDB?

Document-oriented, schema-less, horizontal scaling (sharding), replication (replica sets), rich query language, aggregation pipeline, GridFS for large files, indexing.

3. SQL vs NoSQL?

SQL: structured tables, fixed schema, ACID transactions, vertical scaling. NoSQL: flexible documents/key-value/graph, dynamic schema, horizontal scaling, eventual consistency by default.

4. What is a document in MongoDB?

A single record stored as a BSON (Binary JSON) object. Like a row in SQL but can have nested objects and arrays.

```
{ _id: ObjectId("..."), name: "Faizan", skills: ["React","Django"], age: 22 }
```

5. What is a collection?

A group of documents — equivalent to a table in SQL. No fixed schema, so documents in same collection can have different fields.

6. insertOne() vs insertMany()?

```
db.users.insertOne({ name: "Ali" })              // inserts 1 document
db.users.insertMany([{ name: "A" },{ name: "B" }])  // inserts multiple at once
```

7. What is ObjectId?

MongoDB's default unique identifier for each document. A 12-byte BSON type: 4 bytes timestamp + 5 bytes random + 3 bytes incrementing counter. Automatically added as _id.

8. find() vs findOne()?

```
db.users.find({ age: 22 })      // returns cursor with ALL matching docs
db.users.findOne({ age: 22 })   // returns first matching doc only
```

9. What are indexes in MongoDB?

Data structures that speed up queries. Without index = full collection scan O(n). With index = O(log n). Default index on _id.

```
db.users.createIndex({ email: 1 })   // 1 = ascending, -1 = descending
```

10. What is aggregation in MongoDB?

A pipeline to process and transform documents — like SQL GROUP BY + JOIN + calculations combined.

```
db.orders.aggregate([
  { $match: { status: "paid" } },
  { $group: { _id: "$userId", total: { $sum: "$amount" } } },
  { $sort: { total: -1 } }
])
```

11. $match vs $filter?

$match filters documents at the pipeline stage level (like WHERE). $filter filters elements inside an array field within a document.

12. Normalization vs Denormalization?

Normalization: split data into separate collections, reference by id (like SQL JOINs). Denormalization: embed related data inside the same document. MongoDB favors denormalization for read performance.

13. What is sharding?

Horizontal scaling — splitting data across multiple servers (shards) based on a shard key. Each shard holds a subset of data. Enables massive scale beyond one machine.

14. What is replication?

Keeping copies of data on multiple servers (replica set). One primary accepts writes, secondaries replicate. If primary fails, a secondary is automatically elected primary.

15. How does MongoDB handle transactions?

Multi-document ACID transactions supported since v4.0. Use sessions:

```
const session = client.startSession();
session.startTransaction();
await users.insertOne({ name: "A" }, { session });
await orders.insertOne({ userId: "A" }, { session });
await session.commitTransaction();
```

EXPRESS.JS

1. What is Express.js?

A minimal, unopinionated Node.js web framework. Provides routing, middleware support, and HTTP utilities without enforcing a project structure.

2. Features of Express.js?

Lightweight, fast, middleware-based architecture, robust routing, template engine support, easy REST API creation, large ecosystem.

3. What is app.use()?

Mounts middleware at a path (or globally). Runs for every request matching that path.

```
app.use(express.json())           // parse JSON body for ALL routes
app.use('/api', apiRouter)        // mount router only for /api paths
```

4. What is middleware in Express?

A function with (req, res, next) signature that runs between request and response. Can modify req/res, end the cycle, or call next() to pass to the next middleware.

```
function logger(req, res, next) {
  console.log(req.method, req.path);
  next();   // must call next() or request hangs
}
app.use(logger);
```

5. app.use() vs app.METHOD()?

app.use() runs for ALL HTTP methods. app.get() / app.post() etc. run only for that specific HTTP method.

6. How to handle errors in Express?

```
// error handler must have 4 params (err first)
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ message: err.message });
});
// trigger it by calling next(err) in any route
```

7. What is next() in middleware?

A function that passes control to the next middleware or route handler. Without calling it, the request-response cycle is stuck. next(err) passes to error handler.

8. What is routing in Express?

```
app.get('/users', (req, res) => res.json(users))
app.post('/users', (req, res) => { /* create */ })
app.put('/users/:id', (req, res) => { /* update */ })
app.delete('/users/:id', (req, res) => { /* delete */ })
```

9. How to handle 404 errors?

```
// add AFTER all other routes
app.use((req, res) => {
  res.status(404).json({ message: "Route not found" });
});
```

10. How to parse JSON in Express?

```
app.use(express.json())          // parses JSON body -> req.body
app.use(express.urlencoded({ extended: true }))  // parses form data
```

11. What is CORS and how to enable it?

Cross-Origin Resource Sharing — browser security policy blocking requests from different domains. Enable with cors package:

```
const cors = require('cors');
app.use(cors())                          // allow all origins
app.use(cors({ origin: 'http://localhost:3000' }))  // specific origin
```

12. What is package.json?

Project manifest file. Contains name, version, scripts, dependencies, devDependencies. npm install reads it to install all packages.

13. How to serve static files in Express?

```
app.use(express.static('public'))   // serves files from /public folder
// now http://localhost:3000/image.png serves public/image.png
```

14. res.send() vs res.json()?

res.send() sends any type (string, buffer, object). res.json() always serializes to JSON and sets Content-Type: application/json header automatically.

15. How to implement authentication in Express?

```
const jwt = require('jsonwebtoken');
// Login route - issue token
app.post('/login', (req, res) => {
  const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn: '1d' });
  res.json({ token });
});
// Protect routes with middleware
function auth(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch { res.status(401).json({ message: 'Invalid token' }); }
}
app.get('/profile', auth, (req, res) => res.json(req.user));
```

NODE.JS

1. What is Node.js?

A JavaScript runtime built on Chrome's V8 engine. Runs JS outside the browser, on the server. Single-threaded, event-driven, non-blocking I/O.

2. Features of Node.js?

Non-blocking I/O, event-driven, single-threaded with event loop, fast (V8 engine), npm ecosystem, great for real-time apps and APIs.

3. Node.js vs JavaScript?

JavaScript is the language. Node.js is a runtime environment that executes JS on the server side. Node adds modules like fs, http, path that don't exist in browser JS.

4. What is the event loop?

The mechanism that handles async operations. Node has one thread. Event loop continuously checks the call stack and callback queue — if call stack is empty, it pushes the next callback from the queue.

```
console.log('1');
setTimeout(() => console.log('2'), 0);
console.log('3');
// Output: 1, 3, 2
// setTimeout callback goes to queue, executes after synchronous code
```

5. What is non-blocking I/O?

Node doesn't wait for operations (file read, DB query, network call) to complete. It registers a callback and moves on. When operation finishes, callback is called. This allows handling thousands of concurrent requests.

6. What is package.json?

Project configuration file. Tracks dependencies, scripts (npm start, npm test), project name, version. Run npm init to create it.

7. require() vs import?

require() = CommonJS (older, synchronous, used in Node by default). import = ES Modules (modern, static, used in React/frontend). Node supports both now with .mjs extension or 'type':'module' in package.json.

8. How does Node handle concurrency?

Single thread + event loop + async callbacks. Heavy CPU tasks block the thread — use Worker Threads for those. I/O tasks (DB, network, files) are handled asynchronously via libuv thread pool.

9. What are streams in Node?

Objects for reading/writing data in chunks instead of loading everything into memory. Types: Readable, Writable, Duplex, Transform.

```
const fs = require('fs');
fs.createReadStream('big.txt').pipe(res);  // stream file to response
```

10. What is fs module?

File System module — read, write, delete, rename files and directories.

```
const fs = require('fs');
fs.readFile('file.txt', 'utf8', (err, data) => console.log(data));
fs.writeFileSync('out.txt', 'hello');   // sync version
```

11. process.nextTick() vs setImmediate()?

process.nextTick() runs BEFORE the event loop continues (current operation finishes). setImmediate() runs in the CHECK phase of the next event loop iteration. nextTick has higher priority.

12. What are Buffers?

Raw binary data storage — used when dealing with TCP streams, file system operations, or any binary data. Like arrays of bytes.

```
const buf = Buffer.from('hello', 'utf8');
console.log(buf.toString());  // 'hello'
```

13. How to handle errors in Node?

```
// Async callbacks
fs.readFile('x.txt', (err, data) => {
  if (err) return console.error(err);
});
// Promises
fetchData().catch(err => console.error(err));
// async/await
try { await fetchData(); } catch(err) { console.error(err); }
// Uncaught exceptions
process.on('uncaughtException', err => console.error(err));
```

14. What is clustering in Node?

Node is single-threaded, so only uses 1 CPU core. Cluster module forks multiple child processes (workers) each on their own core. Load is distributed across them.

```
const cluster = require('cluster');
if (cluster.isMaster) {
  require('os').cpus().forEach(() => cluster.fork());
} else {
  require('./server');  // each worker runs the server
}
```

15. What is environment variable?

Variables set outside your code — for secrets, config, API keys. Access via process.env.VAR_NAME. Store in .env file, load with dotenv package.

```
require('dotenv').config();
const port = process.env.PORT || 3000;
const secret = process.env.JWT_SECRET;
```

REACT.JS

1. What is React?

A JavaScript library by Facebook for building user interfaces. Component-based, declarative, uses a virtual DOM for efficient UI updates.

2. Features of React?

Component-based, virtual DOM, one-way data flow, JSX syntax, hooks for state/lifecycle, large ecosystem, React Native for mobile.

3. What is JSX?

JavaScript XML — lets you write HTML-like syntax inside JavaScript. Babel compiles it to React.createElement() calls.

```
const el = <h1>Hello {name}</h1>;
// compiles to:
const el = React.createElement('h1', null, 'Hello ', name);
```

4. What are components in React?

Reusable, independent pieces of UI. Function components (modern) or class components (older). Each manages its own state and renders JSX.

5. Functional vs Class components?

Functional: simpler, use hooks, easier to test, modern standard. Class: older, use lifecycle methods and this.state. React recommends functional + hooks now.

```
// Functional (modern)
function Greeting({ name }) { return <h1>Hi {name}</h1>; }

// Class (older)
class Greeting extends React.Component {
  render() { return <h1>Hi {this.props.name}</h1>; }
}
```

6. What are props?

Properties passed from parent to child component. Read-only — child cannot modify them. One-way data flow.

```
<UserCard name="Faizan" age={22} />
// inside UserCard:
function UserCard({ name, age }) { return <p>{name} is {age}</p>; }
```

7. What is state?

Data that belongs to a component and can change over time. When state changes, React re-renders the component.

```
const [count, setCount] = useState(0);
<button onClick={() => setCount(count + 1)}>Count: {count}</button>
```

8. State vs Props?

Props: passed from parent, read-only, external data. State: managed inside component, mutable, triggers re-render when changed.

9. What is React Router?

Library for client-side routing in React — navigating between pages without full page reload.

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/login" element={<Login />} />
    <Route path="/users/:id" element={<UserProfile />} />
  </Routes>
</BrowserRouter>
```

10. What are Hooks?

Functions that let functional components use state and lifecycle features. Introduced in React 16.8. Rules: only call at top level, only in React functions.

11. Most used React Hooks?

```
useState()      // local state
useEffect()     // side effects (fetch, timers, subscriptions)
useContext()    // consume context without prop drilling
useRef()        // reference DOM elements or persist values
useMemo()       // memoize expensive computed values
useCallback()   // memoize functions to avoid re-creation
useReducer()    // complex state logic (like mini Redux)
```

12. What is the Virtual DOM?

A lightweight in-memory copy of the real DOM. React updates the virtual DOM first, diffs it with previous version (reconciliation), then applies ONLY the changed parts to real DOM. Makes updates fast.

13. Controlled vs Uncontrolled components?

Controlled: form input value managed by React state — React is the single source of truth. Uncontrolled: DOM manages its own state, accessed via refs.

```
// Controlled
<input value={name} onChange={e => setName(e.target.value)} />
```

```
// Uncontrolled
const ref = useRef();
<input ref={ref} />   // access via ref.current.value
```

14. How does useEffect() work?

```
useEffect(() => {
  // runs after render
  fetchData();
  return () => { /* cleanup on unmount */ };
}, [dependency]);    // runs when dependency changes
// [] = run once on mount
// no array = run after every render
// [id] = run when id changes
```

15. What is context in React?

A way to share data globally without prop drilling. Create a context, wrap with Provider, consume with useContext() anywhere in the tree.

```
const AuthContext = createContext();
function App() {
  return <AuthContext.Provider value={{ user, logout }}>
    <Dashboard />
  </AuthContext.Provider>;
}
function Dashboard() {
  const { user } = useContext(AuthContext);
}
```

GENERAL / JAVASCRIPT

1. var vs let vs const?

var: function-scoped, hoisted, can be redeclared — avoid. let: block-scoped, not hoisted, can be reassigned. const: block-scoped, cannot be reassigned (but object properties can change).

2. What are closures?

A function that remembers variables from its outer scope even after the outer function has returned.

```
function counter() {
  let count = 0;
  return function() { return ++count; };
}
const inc = counter();
inc(); // 1
inc(); // 2  -- count is remembered via closure
```

3. What is hoisting?

JavaScript moves variable and function declarations to the top of their scope before execution. var is hoisted (undefined). let/const hoisted but not initialized (TDZ). Functions fully hoisted.

4. What is 'this' keyword?

Refers to the object that called the function. In regular function: depends on call site. In arrow function: inherits 'this' from surrounding scope (lexical this).

5. == vs ===?

== does type coercion before comparing (loose). === compares value AND type without coercion (strict). Always use ===.

```
0 == '0'   // true  (coerced)
0 === '0'  // false (different types)
null == undefined   // true
null === undefined  // false
```

6. What are Promises?

An object representing eventual completion or failure of async operation. States: pending, fulfilled, rejected.

```
fetch('/api/users')
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

7. What is async/await?

Syntactic sugar over Promises — makes async code look synchronous. async function always returns a Promise. await pauses execution until Promise resolves.

```
async function getUsers() {
  try {
    const res = await fetch('/api/users');
    const data = await res.json();
    return data;
  } catch (err) {
    console.error(err);
  }
}
```

8. What is event delegation?

Instead of adding event listener to each child element, add one listener to the parent. Event bubbles up from child to parent. Efficient for dynamic lists.

```
document.getElementById('list').addEventListener('click', (e) => {
  if (e.target.tagName === 'LI') console.log(e.target.textContent);
});
```

9. What is debounce and throttle?

Debounce: delay function execution until user stops triggering (search input — wait until done typing). Throttle: limit function to run at most once per interval (scroll events).

10. Deep copy vs Shallow copy?

Shallow: copies references — nested objects are shared. Deep: copies everything, fully independent.

```
// Shallow
const copy = { ...original };        // nested objects still shared

// Deep
const deep = JSON.parse(JSON.stringify(original));  // simple deep copy
// or use structuredClone(original)  // modern, handles more types
```

FULL STACK / MERN

1. What is the MERN stack?

MongoDB (database) + Express.js (backend framework) + React.js (frontend) + Node.js (runtime). Full JavaScript stack from database to UI.

2. How does data flow in a MERN app?

User interacts with React UI → React sends HTTP request to Express API → Express queries MongoDB via Mongoose → Returns data → React updates UI with state.

3. How do you connect React with Node/Express?

```
// In React (axios or fetch)
const res = await axios.get('http://localhost:5000/api/users');

// In Express
app.get('/api/users', async (req, res) => {
  const users = await User.find();
  res.json(users);
});
// Add CORS in Express: app.use(cors({ origin: 'http://localhost:3000' }))
```

4. How do you connect Express with MongoDB?

```
const mongoose = require('mongoose');
mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log('MongoDB connected'))
  .catch(err => console.log(err));
```

5. How do you handle authentication in MERN?

Register/Login → Express validates credentials → signs JWT → React stores token (localStorage or httpOnly cookie) → sends token in Authorization header with every protected request.

6. How do you implement protected routes in React?

```
function ProtectedRoute({ children }) {
  const token = localStorage.getItem('token');
  return token ? children : <Navigate to="/login" />;
```

```
}
<Route path="/dashboard" element={<ProtectedRoute><Dashboard /></ProtectedRoute>} />
```

7. How do you store JWT token in MERN?

localStorage: easy but vulnerable to XSS. httpOnly cookie: more secure, JS can't access it, server sets it. For production: httpOnly cookie preferred.

8. How do you handle CORS in a MERN app?

```
// Express side
const cors = require('cors');
app.use(cors({ origin: 'http://localhost:3000', credentials: true }));
```

9. How do you deploy a MERN app?

Frontend: build React (npm run build) → deploy to Vercel/Netlify. Backend: deploy Express to Railway/Render/Heroku/EC2. MongoDB: use MongoDB Atlas (cloud). Set environment variables in deployment platform.

10. Folder structure of a MERN project?

```
project/
■■■ client/          # React frontend
■   ■■■ src/
■   ■   ■■■ components/
■   ■   ■■■ pages/
■   ■   ■■■ context/
■   ■   ■■■ services/    # API calls
■■■ server/          # Express backend
■   ■■■ models/          # Mongoose schemas
■   ■■■ routes/          # Express routes
■   ■■■ controllers/     # Route handlers
■   ■■■ middleware/      # auth, error handlers
■   ■■■ index.js
```

11. How do you handle form validation in MERN?

Frontend: libraries like react-hook-form or yup for client-side. Backend: express-validator or joi for server-side. Always validate on BOTH sides — never trust frontend alone.

12. How do you upload files in MERN?

```
// Express with multer
const multer = require('multer');
const upload = multer({ dest: 'uploads/' });
app.post('/upload', upload.single('file'), (req, res) => {
  res.json({ filename: req.file.filename });
});
// React: use FormData
const formData = new FormData();
formData.append('file', selectedFile);
await axios.post('/upload', formData);
```

13. How do you handle real-time data in MERN?

Use Socket.io — WebSocket library. Server emits events, React client listens.

```
// Server
io.on('connection', socket => {
  socket.emit('message', 'Hello client');
  socket.on('send', data => io.emit('message', data));
});
// React
const socket = io('http://localhost:5000');
socket.on('message', data => setMessages([...messages, data]));
```

14. How do you optimize performance in MERN?

React: React.memo, useMemo, useCallback, lazy loading, code splitting. Express: caching (Redis), pagination, DB indexing. MongoDB: proper indexes, aggregation instead of JS loops. General: CDN for static assets, gzip compression.

15. How do you test a MERN application?

React: Jest + React Testing Library. Express: Jest + Supertest for API testing. MongoDB: in-memory mongodb (mongodb-memory-server). E2E: Cypress or Playwright.

TOOLS & CONCEPTS

1. What is REST API?

Representational State Transfer — architectural style for APIs. Uses HTTP methods (GET/POST/PUT/DELETE), stateless, each request is self-contained, resources identified by URLs.

2. What is RESTful architecture?

Client-server separation, stateless, cacheable, uniform interface (consistent URLs + methods), layered system. A RESTful API strictly follows these constraints.

3. What is JWT?

JSON Web Token — a compact, signed token for auth. Three parts: Header.Payload.Signature. Server signs it with secret, client sends it back with each request. Stateless auth — server doesn't store sessions.

```
// Structure (base64 encoded):
eyJhbGciOiJIUzI1NiJ9.eyJpZCI6IjEyMyJ9.HMAC_SIGNATURE
//  HEADER                PAYLOAD           SIGNATURE
```

4. Authentication vs Authorization?

Authentication: WHO are you? (login, verify identity). Authorization: WHAT can you do? (check permissions after auth). You authenticate first, then get authorized to specific resources.

5. What is Postman?

API testing tool. Send HTTP requests (GET/POST/PUT/DELETE), set headers, body, auth tokens. Test your API without needing a frontend. Also supports automated test collections.

6. What is Git and GitHub?

Git: local version control system — track changes, branches, commits. GitHub: cloud hosting for Git repos — collaboration, pull requests, CI/CD integration.

7. What is MVC architecture?

Model (data + DB logic) + View (UI/templates) + Controller (request handling, glues Model and View). Separates concerns for maintainability. Express follows MVC. Django follows MVT (View = Controller, Template = View).

8. What is environment variable?

Config values stored outside code — API keys, DB connection strings, ports, secrets. Never hardcode in source. Use .env file with dotenv in Node, set in deployment platform (Vercel, Railway, etc.)

9. What is CORS?

Cross-Origin Resource Sharing — browser blocks JS requests to different domain/port by default. Server must send Access-Control-Allow-Origin header to permit. Enable in Express with cors() middleware.

10. What is rate limiting?

Restricting number of requests a client can make in a time window. Prevents abuse, DDoS, brute force attacks.

```
const rateLimit = require('express-rate-limit');
app.use(rateLimit({ windowMs: 15 * 60 * 1000, max: 100 }));  // 100 req/15min
```

11. What is API versioning?

Managing multiple versions of an API so old clients don't break when you make changes. Common approach: URL versioning.

```
app.use('/api/v1', v1Router);
app.use('/api/v2', v2Router);
```

12. What is CI/CD?

Continuous Integration / Continuous Deployment. CI: auto-run tests on every code push. CD: auto-deploy to production if tests pass. Tools: GitHub Actions, Jenkins, GitLab CI.

13. What is Docker?

Containerization platform — packages your app + all dependencies into a container that runs identically anywhere. No 'works on my machine' problem.

```
FROM node:18
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "index.js"]
```

14. What is WebSocket?

Full-duplex communication protocol over a single TCP connection. Unlike HTTP, connection stays open — both client and server can send messages anytime. Used for real-time apps (chat, live updates, games). Socket.io is the popular library.

15. What is Redis and why use it?

An in-memory key-value store. Blazing fast because everything is in RAM. Used for: caching (store DB query results), session storage, rate limiting, pub/sub messaging, job queues.

```
const redis = require('redis');
const client = redis.createClient();
await client.set('user:123', JSON.stringify(userData), { EX: 3600 }); // 1hr TTL
const cached = await client.get('user:123');
```